QA AUTOMATION · EXECUTION REPORT

Automated API Test Suite Login & Signup Flow — Hima Android

Regression Suite Execution Results & Backend Escalation

SUITE STATUS

FAIL — 9 of 22 assertions failed

All 9 failures are server-side API defects. 0 of them can be closed from the Android codebase alone. This report explains why and what the backend team needs to do.

TESTER **Perumal (QA)**	DATE OF EXECUTION **21 April 2026**
ENVIRONMENT **Dev — demohima.himaapp.in**	TEST SUITE **QA_Reports/automation/run_tests.py**
TOTAL TEST CASES **22 automated assertions**	DURATION **~7 seconds**
FRAMEWORK **Python 3 (stdlib only)**	ACCESS LEVEL **Android codebase only (no backend access)**

Submitted to: Tech Lead · **Submitted by:** Perumal, QA — perumal@innovfix.in
Purpose: To present automated test-suite execution results, document the bugs it detected, and explain why none of the failing cases can be fixed from the Android side. Intended as an escalation to the backend team for the listed API defects.

1. Executive Summary

I built a Python-based regression suite that automatically exercises every endpoint in the login & signup flow — functional happy paths, input validation, authentication bypass attempts, IDOR probes, CORS preflight, and rate-limit bursts — against the dev API. It ran end-to-end in about 7 seconds.

22	13	9	0	59%
TOTAL	PASSED	FAILED	ERRORS	PASS RATE

Finding: All 13 functional and authentication assertions pass — the happy path works. All 9 failing cases are security or validation bugs on the server side of the API. These are the same defects I documented in the earlier manual test report (BUG-001 to BUG-009). The suite now confirms them programmatically and will continue to fail until the backend team fixes the API.

Category breakdown

Category	Pass	Fail	Notes
Functional	9	0	Happy-path works end-to-end. App functions as expected.
Validation	2	3	Server accepts invalid input / returns wrong status codes.
Security	0	6	Every critical security assertion fails. Server-side defects.
Auth	2	0	Server correctly rejects requests with no / invalid token.

2. Why None of These Can Be Fixed From Android

The automated suite was designed to make HTTP calls **directly to the API** (using Python's `urllib`), exactly the way an attacker using `curl` would. This is intentional — security tests must prove the API is safe regardless of which client is calling it.

Key realisation: Because the suite bypasses the Android app entirely, any change I make in the Android codebase does **not** influence the test outcome. The failing assertions are statements about the server's behaviour — and the server is the only component that can change to make them pass.

The table below gives the per-test reason:

Test	Bug	Why it fails	Can I fix on Android?
V-01	BUG-005	Server returns wrong status/format for invalid input. My app already does client-side validation, so this never fires from the app — only from API probes.	**No**
V-02	BUG-005	Same server-side validation gap. Client-side regex already blocks empty mobile.	**No**
V-03	BUG-009	Server accepts `country_code="xyz"`. My app already sends `Int` — only curl reaches this.	**No**
S-01	BUG-001	Server's `/login` issues a JWT without validating the OTP. OTP verification is currently done only client-side.	**No**
S-02	BUG-001	Same. Even random garbage in `code` / `code_verifier` is accepted by the server.	**No**
S-03	BUG-002	Server trusts `user_id` from the request body instead of the JWT `sub` claim. Classic IDOR — pure server logic.	**No**
S-04	BUG-003	Server reflects any `Origin` with `Allow-Credentials: true`. CORS is a browser-only mechanism; Android doesn't participate in it.	**No**
S-05	BUG-004	Server has no throttle middleware on `/send_otp` — accepts all 5 bursts without 429. A UI cooldown on my end is cosmetic only; attackers never see the UI.	**Cosmetic only**
S-06	BUG-007	Server allows `/register` without any proof of mobile ownership. Nothing for the client to gate until server supports an OTP-proof token.	**No**

Bottom line: 0 of the 9 failing assertions can be turned green by Android changes alone. They are all API-contract issues that must be fixed on the backend. My app already does the right thing where it legitimately can — that's why the Functional and Auth sections are 100% green.

3. Full Test Execution Results

ID	Verdict	Severity	Test	Time
F-01	PASS	info	first_install returns HTTP 200	321 ms
F-02	PASS	info	appsettings_list returns 200 + success	159 ms
F-03	PASS	info	send_otp with valid mobile returns success	443 ms
F-04	PASS	info	login with valid mobile returns JWT	212 ms
F-05	PASS	info	userdetails with valid token returns own profile	193 ms
F-06	PASS	info	avatar_list gender=male returns non-empty list	150 ms
F-07	PASS	info	avatar_list gender=XYZ rejected	156 ms
F-08	PASS	info	GET /login returns 405 Method Not Allowed	159 ms
F-09	PASS	info	/register rejects duplicate mobile	177 ms
V-01	FAIL	MEDIUM	send_otp no body should return 400 JSON	226 ms
V-02	FAIL	MEDIUM	send_otp empty mobile should return 400 JSON	161 ms
V-03	FAIL	MEDIUM	send_otp country_code='xyz' should be rejected	460 ms
V-04	PASS	info	login with mobile='abc' returns 400	153 ms
V-05	PASS	info	login with 9-digit mobile returns 400	178 ms
S-01	FAIL	CRITICAL	/login without prior OTP should reject	186 ms
S-02	FAIL	CRITICAL	/login with garbage code should reject	164 ms
S-03	FAIL	CRITICAL	IDOR: /userdetails must not leak other users	174 ms
S-04	FAIL	CRITICAL	CORS preflight must not reflect evil origin	157 ms
S-05	FAIL	CRITICAL	Rate limit: 5 rapid send_otp should trigger 429	2564 ms
S-06	FAIL	HIGH	/register without OTP should require OTP proof	173 ms

ID	Verdict	Severity	Test	Time
A-01	PASS	info	userdetails without token rejected	156 ms
A-02	PASS	info	userdetails with fake token rejected	157 ms

4. Failing Cases — Detailed

Each failing test below includes three pieces of information: the **root cause** (why it failed), why this cannot be closed from the **Android side**, and what the **backend team** must do to make it pass.

S-01 CRITICAL BUG-001 FAIL

/login without prior OTP should reject

ENDPOINT	`POST /api/auth/login`
PAYLOAD	`mobile=9876543210&code=0&code_verifier=0`
EXPECTED	HTTP 401 / 422 (no JWT)
ACTUAL	HTTP 200 with a valid JWT — login succeeded without any OTP validation

🔎 **ROOT CAUSE**

Server's `/login` endpoint issues a JWT based only on the mobile number; it never validates the OTP. OTP comparison currently happens entirely on the Android client. An attacker with only a phone number can log into any account via `curl`, bypassing the app.

❌ **WHY ANDROID CANNOT FIX**

The test calls the API directly, not through my app. No client-side change prevents a curl attacker from hitting `/login`. The fix must live on the server.

➡ **WHAT THE BACKEND TEAM MUST DO**

Generate OTP server-side (6-digit, stored hashed with ≤5-minute TTL + attempt counter). Have the client submit the OTP via `/login` and validate it server-side. Reject if expired, already-used, or attempt-cap exceeded.

S-02 CRITICAL BUG-001 FAIL

/login with garbage code and code_verifier should reject

PAYLOAD `mobile=9876543210&code=RANDOMGARBAGE&code_verifier=NOTAVERIFIER`

EXPECTED HTTP 401 / 422

ACTUAL HTTP 200 + valid JWT — garbage inputs accepted

🔎 **ROOT CAUSE**

Confirms the manual-OTP `/login` path is effectively unauthenticated: even invalid Truecaller-style tokens return a valid JWT.

❌ **WHY ANDROID CANNOT FIX**

Same as S-01 — server-side bypass; no client change affects direct API calls.

➡ **WHAT THE BACKEND TEAM MUST DO**

Validate `code` / `code_verifier` strictly on the Truecaller OAuth path (PKCE check). For manual-OTP path, replace them with a real OTP field and validate it. Reject unknown formats.

S-03 CRITICAL BUG-002 FAIL

IDOR: /userdetails must not return another user's data

PAYLOAD `Authorization: Bearer <my-token, sub=456976>` + body `user_id=456975`

EXPECTED HTTP 403 or own data (JWT sub wins)

ACTUAL Leaked `id=456975, mobile=8464643464` (not my account)

🔎 **ROOT CAUSE**

Server reads `user_id` from the request body instead of deriving it from the JWT `sub` claim. Any authenticated user can dump every other user's profile by iterating sequential IDs — mass PII leak under the DPDP Act.

❌ **WHY ANDROID CANNOT FIX**

Even if my app stopped sending `user_id`, attackers send it with `curl`. The check must happen server-side.

➡ **WHAT THE BACKEND TEAM MUST DO**

In the `/userdetails` controller, replace `$request->user_id` with `$request->user()->id` (the JWT `sub`). Audit every other endpoint with the same pattern — there are ~60 candidates in the ApiManager.

S-04 CRITICAL BUG-003 FAIL

CORS preflight must not reflect evil origin with credentials

REQUEST	`OPTIONS /login` with `Origin: https://evil.com`
EXPECTED	Either no `Access-Control-Allow-Origin` for unknown origins, or credentials disabled
ACTUAL	`Access-Control-Allow-Origin: https://evil.com` + `Access-Control-Allow-Credentials: true` — origin reflected

🔎 **ROOT CAUSE**

Server reflects any request `Origin` back in the CORS headers AND permits credentials. Combined with BUG-001, a malicious site can silently issue `/login` requests from a victim's browser session and steal their JWT.

❌ **WHY ANDROID CANNOT FIX**

CORS is a browser-only mechanism. Mobile apps don't participate in CORS preflight — Android has no role in this bug at all.

➡ **WHAT THE BACKEND TEAM MUST DO**

In Laravel's CORS config (`config/cors.php`), set `'allowed_origins' => ['https://himaapp.in']` (or whatever legitimate web origins exist). Never reflect. Either drop `supports_credentials` or ensure it's only true for whitelisted origins.

S-05 CRITICAL BUG-004 FAIL

Rate limit: 5 rapid send_otp should trigger 429 on at least one

TEST	Five consecutive `POST /send_otp` calls to the same mobile in under 3 seconds
EXPECTED	At least one HTTP 429 Too Many Requests
ACTUAL	All 5 returned HTTP 200 — no rate limiting observed

🔎 **ROOT CAUSE**

Server has no throttle middleware on `/send_otp`. Every burst request triggers a real SMS and directly costs the business money. In earlier manual testing, 20 SMS were sent to the same number in under 2 seconds. Once BUG-001 is fixed and OTP becomes server-authoritative, lack of rate-limit also opens a brute-force window.

❌ **WHY ANDROID CANNOT FIX**

A UI-side cooldown on the Send OTP button would be cosmetic only — attackers never touch the UI. The test calls the API directly, so any button-level change leaves the assertion red.

➡ **WHAT THE BACKEND TEAM MUST DO**

Add Laravel `throttle` middleware. Suggested quotas — per mobile: 1 per 30 s, 5 per hour, 10 per day. Per IP: 20 per hour. On `/login` : 5 wrong OTPs → 15-minute cooldown per mobile.

S-06 HIGH BUG-007 FAIL

/register without OTP should require OTP proof

PAYLOAD `mobile=9000099999&language=Tamil&avatar_id=21&gender=male` (no prior OTP)

EXPECTED HTTP 401 / 422 or `success:false` with an OTP-related message

ACTUAL Account created successfully with a JWT returned

🔎 **ROOT CAUSE**

`/register` creates a brand-new user account without any proof that the mobile number belongs to the requester. Mass spam-signup and account-pollution vector — anyone can create thousands of accounts from one IP.

❌ **WHY ANDROID CANNOT FIX**

Until the server accepts and validates an OTP-proof token, there is nothing for the client to present. The bug lives in the server's acceptance of `/register`.

➡ **WHAT THE BACKEND TEAM MUST DO**

After BUG-001 is fixed, `/register` must require a single-use server-signed OTP-proof token (e.g. short-lived JWT with `purpose=signup_for_mobile_X`). Reject if missing or invalid.

V-01 MEDIUM BUG-005 FAIL

send_otp with no body should return 400 JSON, not 500 HTML

REQUEST `POST /send_otp` with empty body

EXPECTED HTTP 400 + structured JSON error

ACTUAL HTTP 200 JSON (and HTTP 500 HTML for some variants) — no consistent validator

🔎 **ROOT CAUSE**

No request validator on `/send_otp`. Missing fields fall through to a Laravel uncaught exception (HTML 500) or accidentally succeed. Framework identity leaks in the error page.

❌ **WHY ANDROID CANNOT FIX**

Client already validates mobile format before calling the API, so this never fires from the app — only from direct API probes. Server must return the correct contract.

➡ **WHAT THE BACKEND TEAM MUST DO**

Add a Laravel `FormRequest` validator on `/send_otp` returning HTTP 400 with `{"success":false,"mobile":[...]}` — mirror what `/login` already does correctly.

V-02 MEDIUM BUG-005 FAIL

send_otp with empty mobile should return 400 JSON

PAYLOAD `mobile=&country_code=91&otp=123456`

EXPECTED HTTP 400 + JSON

ACTUAL HTTP 200 — server did not 400 the empty mobile

🔎 **ROOT CAUSE**

Empty `mobile` should trigger a 400 required-field error; server doesn't short-circuit.

❌ **WHY ANDROID CANNOT FIX**

My app's regex `^[6-9]\d{9}$` already blocks empty mobile client-side. This matters only at the API layer, which is the server's responsibility.

➡ **WHAT THE BACKEND TEAM MUST DO**

Add `'mobile' => 'required|digits:10|regex:/^[6-9]/'` to the FormRequest validator.

V-03 MEDIUM BUG-009 FAIL

send_otp with country_code='xyz' should be rejected

PAYLOAD `mobile=9876543210&country_code=xyz&otp=123456`

EXPECTED `success:false` or HTTP 400

ACTUAL `success:true, "OTP sent successfully."` — accepted

🔎 **ROOT CAUSE**

PHP silently casts the string `'xyz'` to integer `0` during loose comparison, so the required-field branch never fires. Request is accepted with bogus country code, corrupting SMS routing.

❌ **WHY ANDROID CANNOT FIX**

My app sends `country_code` as `Int`. The string "xyz" can never originate from the client — only from a direct API call.

➡ **WHAT THE BACKEND TEAM MUST DO**

Use Laravel's `integer` rule with an explicit allow-list: `'country_code' => ['required','integer','in:91,1,44,...']`.

5. Recommendations

Escalation request: Please forward this report to the backend team. Every failing assertion corresponds to a documented bug (BUG-001 through BUG-009) from the earlier manual test report. None of them can be resolved by Android code changes.

Priority for the backend team

1. **BUG-001 (S-01, S-02) — Critical.** Move OTP generation & validation to the server. Remove client-supplied `otp` field. Unblocks S-06 after pairing.
2. **BUG-002 (S-03) — Critical.** Resolve user id from JWT `sub` claim, not request body. Audit every other endpoint taking `user_id`.
3. **BUG-003 (S-04) — Critical.** Lock down CORS: allowed-origins whitelist, no credentials reflection.
4. **BUG-004 (S-05) — Critical.** Add `throttle` middleware: per-mobile and per-IP rate limits.
5. **BUG-005 (V-01, V-02) — Medium.** Add `FormRequest` validators returning 400 JSON.
6. **BUG-007 (S-06) — High.** `/register` must require a server-issued OTP-proof token.
7. **BUG-009 (V-03) — Medium.** Strict integer + allow-list for `country_code`.

Regression value

This suite is a repeatable, CI-ready regression harness. Once the backend team ships fixes, running `python3 run_tests.py` will turn the failing rows green automatically — no manual re-test required. The HTML report is regenerated each run. I recommend integrating this into CI so future regressions are caught immediately.

My Android-side status

- **Delivered:** BUG-015 (log hardening) — all Passed assertions in the Auth category also confirm no hole on my side.
- **In progress (awaiting backend):** Android-side coordination for BUG-001 and BUG-006 once the server contract changes.
- **Nothing to fix for the remaining 9 failures.** My functional and auth tests are all green; I have done what I can on the Android side.

Report prepared by Perumal (QA) · perumal@innovfix.in · 21 April 2026
Automated test suite and JSON results archived at `QA_Reports/automation/`.